

Excercises Lab 5 Bonus

Mathematics

1 Compute $137^{51} \pmod{1337}$ using CRT

To calculate $137^{51} \pmod{1337}$ using the Chinese Remainder Theorem (CRT), we first factor the modulus $M = 1337$.

$$1337 = 7 \times 191$$

Since 7 and 191 are coprime primes, we can set up the system of congruences by reducing 137^{51} by each prime factor.

Step 1: Reduce the target number

1. Modulo 7:

$$137 \equiv 4 \pmod{7} \quad (\text{since } 137 = 19 \times 7 + 4)$$

We need to compute $4^{51} \pmod{7}$. By Fermat's Little Theorem, $4^6 \equiv 1 \pmod{7}$. We write the exponent 51 as $6 \times 8 + 3$:

$$4^{51} \equiv (4^6)^8 \cdot 4^3 \equiv 1^8 \cdot 64 \pmod{7}$$

Since $64 = 9 \times 7 + 1$:

$$4^{51} \equiv 1 \pmod{7}$$

2. Modulo 191: We need to compute $137^{51} \pmod{191}$. Using the method of repeated squaring:

$$137^1 \equiv 137 \pmod{191}$$

$$137^2 \equiv 18769 \equiv 51 \pmod{191}$$

$$137^4 \equiv 51^2 = 2601 \equiv 118 \pmod{191}$$

$$137^8 \equiv 118^2 = 13924 \equiv 172 \pmod{191}$$

$$137^{16} \equiv 172^2 = 29584 \equiv 170 \pmod{191}$$

$$137^{32} \equiv 170^2 = 28900 \equiv 59 \pmod{191}$$

Combining the powers for $51 = 32 + 16 + 2 + 1$:

$$137^{51} \equiv 137^{32} \cdot 137^{16} \cdot 137^2 \cdot 137^1 \pmod{191}$$

$$137^{51} \equiv 59 \cdot 170 \cdot 51 \cdot 137 \pmod{191}$$

$$137^{51} \equiv 10030 \cdot 6987 \pmod{191}$$

Reducing intermediate products ($10030 \equiv 98$ and $6987 \equiv 111$):

$$137^{51} \equiv 98 \cdot 111 = 10878 \equiv 182 \pmod{191}$$

We now solve the system:

$$x \equiv 1 \pmod{7}$$

$$x \equiv 182 \pmod{191}$$

Step 2: Apply the CRT formula

The solution is $x = \sum a_i M_i y_i \pmod{M}$, where $M_i = M/n_i$ and $y_i \equiv M_i^{-1} \pmod{n_i}$.

1. For $n_1 = 7$:

$$M_1 = \frac{1337}{7} = 191$$

We find the modular inverse of 191 $\pmod{7}$:

$$191 \equiv 2 \pmod{7}$$

$$2y_1 \equiv 1 \pmod{7} \implies y_1 = 4$$

2. For $n_2 = 191$:

$$M_2 = \frac{1337}{191} = 7$$

We find the modular inverse of 7 $\pmod{191}$ using the Extended Euclidean Algorithm:

$$191 = 27(7) + 2$$

$$7 = 3(2) + 1$$

Back-substitution:

$$1 = 7 - 3(2) = 7 - 3(191 - 27(7)) = 82(7) - 3(191)$$

Thus, $7(82) \equiv 1 \pmod{191}$, so $y_2 = 82$.

Step 3: Compute final sum

$$x = (a_1 M_1 y_1 + a_2 M_2 y_2) \pmod{1337}$$

$$x = (1 \cdot 191 \cdot 4 + 182 \cdot 7 \cdot 82) \pmod{1337}$$

$$x = (764 + 182 \cdot 574) \pmod{1337}$$

$$x = (764 + 104468) \pmod{1337}$$

$$x = 105232 \pmod{1337}$$

$$x = 946$$

Result: $137^{51} \equiv 946 \pmod{1337}$.

2 Prime Generation Function (Sieve of Eratosthenes)

The following Python function implements the Sieve of Eratosthenes to return all prime numbers up to a parameter n .

Listing 1: Sieve of Eratosthenes

```
def get_primes_upto(n):  
  
    if n < 2:  
        return []  
  
    # Initialize boolean array  
    is_prime = [True] * (n + 1)  
    is_prime[0] = is_prime[1] = False  
  
    p = 2  
    while (p * p <= n):  
        # If is_prime[p] is not changed, then it is a prime  
        if is_prime[p]:  
            # Update all multiples of p  
            for i in range(p * p, n + 1, p):  
                is_prime[i] = False  
            p += 1  
  
    # Collect primes  
    return [x for x, prime in enumerate(is_prime) if prime]
```

3 CRT for Non-Coprime Moduli

When moduli are not coprime, a solution exists if and only if $a_1 \equiv a_2 \pmod{\gcd(n_1, n_2)}$. The following function implements the Generalized Chinese Remainder Theorem.

Listing 2: Generalized CRT Implementation

```
def extended_gcd(a, b):
    if a == 0:
        return b, 0, 1
    g, y, x = extended_gcd(b % a, a)
    return g, x - (b // a) * y, y

def solve_generalized_crt(congruences):

    if not congruences:
        return None

    # Start with the first equation
    cur_a, cur_n = congruences[0]

    for a, n in congruences[1:]:

        g, p, q = extended_gcd(cur_n, n)

        # Check condition for existence of solution
        if (a - cur_a) % g != 0:
            return None # No solution

        lcm_n = (cur_n * n) // g
        scale = (a - cur_a) // g

        k = (p * scale) % (n // g)

        # Update current x and current modulus
        cur_a = (cur_a + cur_n * k) % lcm_n
        cur_n = lcm_n

    return cur_a, cur_n
```

4 Primality Testing Functions

Below are two implementations of primality testing: a naive trial division and an optimized version using the $6k \pm 1$ rule.

Listing 3: Primality Tests

```
import math

def is_prime_naive(n):

    if n <= 1:
        return False
    # Check every integer from 2 to sqrt(n)
    for i in range(2, int(math.isqrt(n)) + 1):
        if n % i == 0:
            return False
    return True

def is_prime_optimized(n):

    if n <= 1: return False
    if n <= 3: return True
    if n % 2 == 0 or n % 3 == 0: return False

    # Check for divisors of form 6k +/- 1
    i = 5
    while i * i <= n:
        if n % i == 0 or n % (i + 2) == 0:
            return False
        i += 6
    return True
```